NOMENCLATURAS

Nomes de Campos

- Regras:
  Não possuem hífen, nem subscrito
  São escritos em letras maiúsculas
  Possuem apenas letras e números
  Sempre se iniciam por uma letra

- Deve ser iniciado de acordo com a seguinte tabela:

Categoria	Abreviação	Exemplo
Número	NUM	NUMNF = Número da Nota Fiscal
Código	COD	CODFORN = Código Fornecedor
Valor	VAL	VALUNITITEM = Valor Unitário do Item
Data	DAT	DATCADAST = Data de Cadastramento
Status	STA	STAITEM = Status do Item
Nome	NOM	NOMFANTASCLI = Nome Fantasia do Cliente
Descrição	DES	DESMAT = Descrição do Material
Tipo	TIP	TIPNF = Tipo de Nota Fiscal
Endereço	END	ENDFORN = Endereço do Fornecedor
Telefone	TEL	TELFORN = Telefone do Fornecedor
Fax	FAX	FAXFORN = Fax do Fornecedor
Quantidade	QTD	QTDITEM = Quantidade do Item
CEP	CEP	CEPCLI = CEP do Cliente
Razão Social	RAZSOC	RAZSOCCLI = Razão Social do Cliente

Nomes de Variáveis

Eliminar variáveis não criadas e usadas

Verificar se todas as variáveis foram inicializadas (**cuidado com variáveis globais**)

- O nome das variáveis não devem conter hífens, nem subscrito
- Todos os nomes de variáveis devem ser escritos em notação húngara
- As variáveis devem iniciar com a primeira letra (às vezes, a segunda letra também) minúscula, representado o seu tipo (ver tabela a seguir)
- As variáveis locais de uma procedure ou function **NÃO** devem começar jamais pela letra F maiúscula. Reservamos a letra F maiúscula para nomes de variáveis que são propiedades de formulários (ou seja, apenas variáveis declaradas dentro das seções private ou public de TForms é que devem iniciar pela letra F maiúscula). Como nenhum dos tipos de variáveis utilizadas na Informatec começa com "f", nenhuma variável começará com essa letra.

Tipo	Exemplo de Nome

Extended	eValorUnit
Integer	iContador
String	sTexto
TDateTime	dtDataFatura
Array	aMeses
Boolean	bStatus

Nomes de Propriedades (variáveis) de formulários

- O nome das propriedades (“variáveis” ou “campos” de uma classe, definidas nas seções **public** ou **private**) devem seguir as mesmas regras da nomeação de variáveis comuns, com a diferença de que a primeira letra do nome deve ser “F” (de Field):

Propiedade
FsStatus : String
FiContador: Integer
FbContinua: Boolean

Nomes de Componentes

- Os nomes dos componentes devem iniciar com o próprio nome pré-definido pelo Delphi, seguido por uma palavra complementar – em notação húngara – que especifique sua função dentro do módulo.
- Se o componente estiver ligado a campos de tabelas, a palavra complementar deve ser escrita em maiúsculas, com o mesmo nome do campo da tabela.

Componente	**Exemplo**
Button	ButtonImprimir
BitBtn	BitBtnFehar
ComboBox	ComboBoxEstados
ListBox	ListBoxItens
Edit	EditPesquisa
DBEdit	DBEditCODFORN
MaskEdit	MaskEditDESPRODUTO

Nomes de Campos dos Componentes TQuery ou ww TQuery (“campos persistentes”)

- Se a Query (ou wwQuery) tiver finalidade genérica (inserções, atualizações,deleções ou outro processamento qualquer), e sua propriedade “SQL” varia e acordo com a rotina do módulo, deve-se usar a palavra complementar “Aux” ou “Auxiliar”:
    - QueryAux
- Se a Query (ou wwQuery) tiver finalidade de consulta em diversas tabelas, pode-se utilizar o nome da tabela principal para atribuir um nome ao componente Query, ou usar a denominação “Consulta”:
    - QueryConsulta
- Se a Query (ou wwQuery) for utilizada para realizar atualizações diversas no banco de dados, pode-se utilizar a palavra complementar “Atualizacao” para atribuir um nome ao componente:
    - QueryAtualizacao (sem acentos)

Nomes de Tabelas

- Os nomes das tabelas do banco de dados devem iniciar-se com ”TB”, seguidas de quatro letras que identifiquem o sistema ao qual pertencem, e um número seqüencial com três posições. Portanto, as tabelas devem ter até 9 caracteres em seu nome. Por exemplo:
    - TBGENE001 = Unidades de Medida (tabela usada por vários sistemas)
    - TBFATU010 = Pedido de Vendas
    - TBCOMP001 = Fornecedores
- Tabelas locais, utilizadas para processamento temporário (principalmente para preparação de dados a serem exibidos em relatórios) devem iniciar-se pelas letras “TBTMP”, seguidas de um número seqüencial. Por exemplo:
    - TBTMP001, TBTMP002, ETC.

Nomes de Módulos

- O nome do módulo deve iniciar com letras que identifiquem o sistema ao qual pertence, como por exemplo:
    - COMP001 = Sistema de Compras
    - FATU007 = Sistema de Faturamento
    - ESTO010 = Sistema de Estoque

- Se o módulo tem finalidade de uso em diversos sistemas, utilizar as iniciais **“GENE”** (genérico), como por exemplo:
    - GENE001 = Cadastro de Unidades de Medida

- Se o módulo for tipo “Pesquisa”, deve iniciar-se com as letras “PESQ”, seguidas de uma abreviação que indiquem qual é a pesquisa. Por exemplo:
    - PESQUM = Pesquisa de Unidades de Medida
    - PESQCLI = Pesquisa de Clientes

Atenção: Não utilizar mais de 8 caracteres no nome do módulo.

VARIÁVEIS

- Não utilizar variáveis desnecessariamente.
- Atentar-se para códigos copiados – eliminar o que é desnecessário.
- Verificar nos "warnings" da compilação se há variável criada e sem uso.
- Verificar se todas as variáveis foram inicializadas (**cuidado com variáveis globais**)

IDENTAÇÃO

- Usar **3 espaços;**
- Não usar o caracter de tabulação para marcar identação. É possível configurar a IDE para que a tecla TAB ponha 3 caracteres invés do caractere de tabulação;
- Usar Begin..End quando o bloco tiver mais de uma linha;
- O Begin..End deve estar alinhado com o inicio do bloco, não é necessário a identação. Exemplo:

```
If Condição then
Begin
   Comandos
   Comandos
   While not EOF do
   Begin
      Next;
   End;
End;
```

RELATÓRIOS

Dados do Cabeçalho

- Cabeçalho da página – Todos os filtros e opções devem estar representados no cabeçalho da página.

- Quando o relatório possui filtros de intervalo de código, data, etc., o intervalo deve estar claramente representado.

Exemplo: quando existe um filtro de código do material, se o usuário preencher apenas o código inicial, deve ser impresso no cabeçalho: "Código do material a partir de: 9999999". Se forem preenchidos os códigos inicial e final, deve aparecer: "Código do material de 99999 até 99999".

Geração do Arquivo Texto

- O relatório impresso e o arquivo texto devem "bater" entre si.
- O cabeçalho da página deve aparecer apenas uma vez, no começo do arquivo.
- Não devem existir totais.

Valores dos Relatórios

- Quando se trata de valores totais (total da mercadoria, total da nota, etc.), como regra geral devem ser utilizadas 2 casas decimais.
- Para valores unitários, existe um padrão para o Módulo de Compras, onde temos um parâmetro que controla a quantidade de casas. Da mesma forma, para o Faturamento existe um parâmetro que controla o valor unitário (e que pode variar de uma empresa para outra). No Estoque e em Produção, deve ser feita a consulta nas telas e relatórios correlatos , mas em geral tratam-se de quatro casas decimais.
- Para as quantidades, em Compras são utilizadas 5 casas decimais, no Faturamento são utilizadas 3 casas, no Estoque e na Produção 4 casas. Quando unir dados de várias fontes, considerar a origem que possua mais casas.
- Todas as colunas de valores e quantidade devem ter totais (exceto quando se tratar de valor unitário ou média). Qualquer exceção deve estar claramente identificada na definição ou no modelo do relatório.
- Sempre que existir totais, deve ser feita a conferência de que o total "bate" com os valores demonstrados na mesma coluna.
- Nos casos de campos do tipo "data", quando o campo estiver nulo, tratar a impressão, caso contrário será impresso "30/12/1899" (valor padrão quando a data não está preenchida).

<u>Rotinas Padrão</u>

- No Create do Form:
  - Rotina "FormataCampos".(para trazer máscaras dos códigos parametrizados)
  - Rotina "SelecionaParametros" (para trazer os campos das tabelas de parâmetros: TBGENE035, TBCOMP998, etc.)

- Nos botões de pesquisa genérica (lanterna)
  - Utilizar a função "PesqGene", que agrupa o código para acesso de todas as pesquisas parametrizadas que venham a ser utilizadas na tela.

- No clique do botão para imprimir o relatório:
  - Rotina "ValidaDados" (para verificar os intervalos informados na tela)
  - Rotina "ObtemDados" (para trazer as informações – SELECT)
  - Rotina "DefineDestino" (para definir o destino o relatório: tela, impressora ou arquivo).

- Eventos utilizados no componente "ReportSystem" ou "TextFiler"
  - Evento "BeforePrint": código para ajustar tamanho de página, etc.
  - Evento "OverridePreview"(relatório gráfico): código para acessar a tela de visualização do relatório.
  - Evento "PrintHeader": código para chamar a rotina de impressão do cabeçalho da página.
  - Evento "Print": código para chamar a rotina de impressão do cabeçalho da quebra, rotina de impressão dos dados da linha detalhe e rotina para impressão dos totais.

  - Query.First/ProcessMessages/Query.Next

```
QueryRelatorio.Open;
QueryRelatorio.First;
```

```
while not QueryRelatorio.Eof do
begin
   Application.ProcessMessages;
   < comandos >
   QueryRelatorio.Next;
end;
```

Estética do Relatório

- Quando existe conversão de unidades de medida e/ou de moedas (valores), o cálculo deve ser feito para pelo menos 3 dias diferentes (por causa da cotação) ou pelo menos três unidades de medida ou materiais diferentes (por causa do fator de conversão)
- Utilizar apenas uma rotina para o formato texto e o formato gráfico.
- Verificar se não existem bordas “quebradas” e as tabulações estão alinhadas. (utilizando todas as combinações de opções).
- Nas totalizações que possuam códigos, no cabeçalho devem ser impressos o código e o nome. Por exemplo: na quebra por vendedor, devem ser impressos o código e o nome do vendedor.
- Na linha detalhe deve ser feita a conferência de que dados que ultrapassem o espaço estipulado não “empurrem” os dados subseqüentes nem sejam truncados (valores).
- Quando imprimir códigos que possuam máscara parametrizada (por exemplo, código do material) a informação deve ser impressa formatada por este código.

Quebra de Página

Dentro de TODOS os whiles que imprimem dados, testar a quebra de página:

```
QueryRelatorioMestre.Open;
QueryRelatorioMestre.First;
while not QueryRelatorioMestre.Eof do
begin
   Application.ProcessMessages;
   if linesleft < 6 then
   begin
      Inc(iNumPagina);
      ImprimeCabecalhoDaPagina;
      < comandos >
   end;
   < comandos >
   // Impressao dos detalhes:
   QueryRelatorioDetalhes.Close;
   QueryRelatorioDetalhes.ParamByName('CODEMP').AsString :=
QueryRelatorioMestre.FieldByName('CODEMP').AsString;
   QueryRelatorioDetalhes.Open;
  while not QueryRelatorioDetalhes.Eof do
   begin
      if linesleft < 6 then
      begin
         Inc(iNumPagina);
         ImprimeCabecalhoDaPagina;
         ImprimeCabecalhoDosDetalhes;
        comandos >
      end;
    end;
     QueryRelatorioMestre.Next;
end;
```

Relatórios Gráficos (ReportSystem)

- Na rotina que define o destino do relatório(supondo que o nome do módulo, por exemplo, seja RELCB013):

```
case RadioGroupImpressao.ItemIndex of
  0: begin // impressora
end; // 0:
    1: begin // preview
  ReportSystemRelatorio.DefaultDest := rdPreview;
  ReportSystemRelatorio.SystemFiler.FileName := NomeDeArquivoTemporario
(IncluiOuEliminaBarraFinal(CaminhoArqSistema('TEMP'),'E'), 'RELPEDVEN', 'RPT');
  ReportSystemRelatorio.SystemFiler.StreamMode := smFile;

        end;
```

PROCEDURES E FUNÇÕES

- Utilizar a query padrão para Parâmetros (wwqueryparametros).
- A validação dos campos deve ser feita no evento onValidate;.
- Dar preferência ao evento onCreate ao invés do evento onActivate, para garantir que o processamento seja realizado apenas uma vez;
- No evento onCreate configurar:
- - Colocar o nome de exibição da filial antes do título da janela;
- - Formatar os campos da tela;
- - Abrir as queries. Em grandes cadastros, abrir apenas as queries da primeira aba;
- - Desabilitar/Habilitar campos da tela;
- - Configurar os botões do navigator;
- No evento onActivate configurar apenas o foco inicial ao abrir a tela;
- No evento onNewRecord colocar o valor default dos campos;
- Nos eventos BeforeEdit, BeforeInsert e BeforeDelete validar a permissão do usuário;
- Quando existir algum loop de queries que possuam eventos ligados, desligar os eventos antes de começar os loops;
- Quando uma tela chamar outra, as permissões desta nova tela deverão ser carregadas. Ao terminar o processamento da ultima tela, as permissões da tela anterior devem ser carregadas novamente;
- Tratar as exceções ocorridas durante o cadastro nos eventos Error da query;
- Não utilizar DisableControls no DataSet quando o mesmo for o mestre de um mestre-detalhe;
- No evento FormClose, fechar as queries;
- Não usar a rotina “Locate” das Queries (degrada a performance).
- Não acessar campos utilizando Value.
- Evitar fazer a alteração da “String” da Query utilizando o índice da linha. Por exemplo:
- Query.Strings[i] := ‘ WHERE .....’
- ~~Não~~ Evitar criar subrotinas dentro da própria rotina.
- Só utilizar variáveis globais se for extremamente necessário.
- Quando criar componentes em tempo de execução, proteger a criação e liberação de memória com um bloco try...finally.
- Exemplo:

```
Query1 := TQuery.Create(nil);

Try
        { Utilização da Query1 }
        Finally
        Query1.Close;
        Query1.Free;
        End;
```

- No meio de um processamento que possa demorar alguns segundos, utilizar o comando Application.ProcessMessages para atualizar a tela (evita que a tela fique “em branco” enquanto o processamento é feito.

- Não utilizar hífens, nem subscrito nos nomes de procedures e funções
- Procedures e Funções de uso genérico devem ser incluídas no módulo PROCFUNC.PAS
- No local de declaração de procedures e funções deve-se separá-las em dois grupos (procedures e funções) e declará-las em ordem alfabética, dentro de cada grupo.
- ~~Não~~ Evitar criar procedures e funções dentro de outras procedures ou funções . Declarar rotinas dentro de outras rotinas tem dificultado alterações nos módulos.
- Deve-se procurar declarar procedures e funções no próprio módulo em que estas são utilizadas:
- Se o módulo possui um formulário, deve-se declarar a procedure/função na parte **PRIVATE** do formulário. Ou seja, todas as procedures/funções de um módulo que possuem formulário devem pertencer à classe do formulário. Só declararemos procedures/funções fora de um formulário se o módulo **NÃO POSSUI** formulário.
- Se for inevitável que a procedure/função seja utilizada de fora do módulo do formulário, deve-se declara-lá na parte **PUBLIC** do formulário.
- Comentar código não utilizado, evitando duplicar um bloco grande de código. Neste caso, colocar o nome da definição, o nome do programador, a data em que a alteração foi feita e o motivo.
- Quando a rotina ou função chamada estiver no mesmo PAS, utilizar o radical “Self” para identificar esta situação
- Quando a rotina ou função chamada estiver em outro PAS, colocar na frente o nome do arquivo PAS ao qual ele pertence.
- Atribuição de Valores a Campos de Queries
  Deve-se evitar a alteração de campos através dos componentes a eles ligados. Por exemplo, se em um módulo existe o campo QueryTBCOMP002CODUNIMED, que é apresentado na tela através do componente DBEditVALTOTPED, deve-se alterar o conteúdo do campo do seguinte modo (por exemplo):

```
QueryTBCOMP002CODUNIMED.Value: = “PC”;
```

  Deve-se evitar, portanto, alimentar o campo através do componente DBEdit:

```
DBEditCODUNIMED.Text: = “PC”; (evitar este uso)
```

- **Imediatamente ANTES de alimentar os parâmetros, usar o Close:**

```
Query1.Open;
Query1.First;
While not Query1.Eof do
Begin
Application.ProcessMessages;
< comandos >
QuerySaldos.Close;
QuerySaldos.ParamByName(‘ParCodigo’).AsString := Query1.FieldByName(‘CODIGO’).AsString;
QuerySaldos.Open;
<comandos>
Query1.Next;
End;
```

TRANSAÇÕES

- A mensagem de sucesso ou não da transação deve ser incluída após a conclusão da transação;
- Uma transação não pode ter uma tela com intervenção do usuário entre o inicio e fim.
- Todas as validações devem ser feitas antes de iniciar a transação;
- As rotinas de validação de um processamento devem ser colocadas em um procedimento separado do procedimento que realizará o processamento, para que esta validação possa ocorrer fora da transação. Não deve existir uma rotina que utilize estes dois procedimentos em seqüência, pois a transação apenas deve ser iniciada depois que a validação foi feita com sucesso;
- Deve-se garantir que todas as mensagens existentes durante o processamento sejam exibidas apenas após o término da transação;
- Tentar não utilizar o teste se a transação está aberta antes de dar o commit ou rollback, pois este teste pode esconder falhas graves de transação.
- O correto é validar se existe a transação e alimentar uma variável antes de abrir a transação. Antes de dar commit ou rollback, é necessário validar se a transação foi aberta por esta rotina ou não. Se a transação foi aberta pela mesma rotina o encerramento pode ocorrer naturalmente. Caso contrário, o encerramento da transação é de responsabilidade da rotina que a abriu. Exemplo:

```
...
Var bAbrirTransacao : boolean;
Begin
      bAbrirTransacao  := not FormPrincipal.DatabaseSistema.InTransaction;

      if bAbrirTransacao then
         FormPrincipal.DatabaseSistema.StartTransaction;

       Try
            …
            …
            if bAbrirTransacao then
               FormPrincipal.DatabaseSistema.Commit;

        Except
           if bAbrirTransacao then
              FormPrincipal.DatabaseSistema.Rollback;
         End;
End;
```

PERFORMANCE

- Evitar acessos desnecessários ao BD
- Não utilizar arquivos temporários do tipo “DBF”
- **Utilizar ClientDataSet sempre que possível**
- Criar tabelas temporárias diretamente no BD, garantindo que elas sejam únicas por usuário.
- Nas cláusulas SELECT utilizadas, atentar para:
  - As condições mais exclusivas devem ser as primeiras do SELECT;
  - Nos relacionamentos entre as tabelas, colocar os campos na mesma ordem em que eles aparecem nas chaves ou índices.

- Nos relacionamentos entre tabelas, colocar do lado esquerdo a tabela que acompanha a tabela que está do lado direito.

- Em casos em que existe muito volume de dados, criar índices e verificar se houve melhora no desempenho.
- No Delphi novo, quando faz um SELECT 0 AS CAMPO, o novo componente de conexão vai entender o campo como um tipo diferente, e vai dar erro ao criar o TField.

  Por favor, passem a usar CAST(0 AS DECIMAL(28,14)) AS CAMPO, para evitar esse problema.

LÓGICA

- A rotina ou função deve ter alta coesão e baixo acoplamento, ou seja, deve ter um único propósito (a rotina não pode executar várias tarefas totalmente distintas) e deve ser o mais independente possível (não deve haver muitas rotinas que chamem subrotinas, as quais chamam subrotinas, etc.). Estas técnicas visam facilitar o reuso do código.

- Fazer o fluxograma ou pseudocódigo dos trechos do programa que se refere ao processamento que não sofre intervenção do usuário antes de iniciar a codificação do programa. Antes da codificação este fluxograma ou pseudocódigo deve ser validado pelo analista.

TRATAMENTO DE EXCESSÕES

- Antes de cada divisão certificar-se que não resultará em divisão por zero – mesmo que o divisor teoricamente nunca esteja zerado.
- Tratamento de campos nulos

MENSAGENS

  - Deve ter clareza: o usuário deve entender o motivo da mensagem
  - Deve estar gramaticalmente correta: sem erros de português ou sintaxe

COMENTÁRIOS

- Procurar usar sempre comentários nas rotinas menos óbvias, tendo em mente que pode ser necessário que um outro programador tenha de realizar a manutenção do módulo.
- O comentário deve ser feito com o objetivo de esclarecer, **PRINCIPALMENTE AO OUTRO PROGRAMADOR** que venha a ler o código.
- Sempre que possível, utilizar o símbolo "//" para realizar comentários – principalmente quando ocupa apenas uma linha.
- Colocar como comentário: o nome do arquivo da definição, o nome do programador e a data da alteração.
- Evitar deixar grandes blocos comentados no meio de um bloco em uso (como campos de uma query, por exemplo).

- Ter como hábito colocar no final da rotina ou função (após o end que encerra o bloco) um comentário contendo o cabeçalho da rotina/função. O objetivo é identificar facilmente que a rotina ou função se encerra naquela linha.
- Ao chamar uma rotina ou função, colocar comentário com um resumo do processamento que será feito nesta rotina ou função.

Padrão Mobile

1. Variáveis

1.1. Tipagem

O Javascript é uma linguagem de tipagem fraca. No entanto, várias das nossas aplicações, tanto no frontend como no backend, temos usado typescrypt, que ajuda a organizar o código. Quando criarmos variáveis cuja intenção é receber um valor de um tipo definido, devemos utilizar o primeiro (ou primeiros caracteres) para identificar o tipo de dado (semelhante ao padrão usado no delphi)

Tipo	Exemplo de Nome
number	nValorUnit: number;
string	sTexto: string = '';
Date	dtDataFatura = new Date();
array	aMeses: any[] = [];
boolean	bPrimeiroAcesso : boolean

objeto	oPostagem = {
	codigocliente: null,
	postagem: '',
	arquivo: '',
	}

atributos de um objeto	Vamos padronizar os atributos sempre com letra minúscula e sem caractere identificador de tipagem, como no exemplo acima.

Funções armazenadas em variável	fSomaMateriais = (valo1, valor2) => {
	return valo1 + valor2
	}

*para variáveis number que receberão valores inteiros, apesar de não haver distinção no typescript, podemos usar a notação inicial i, para melhor visualização. Por exemplo em um laço iContador.

1.2. Escopo

Sempre priorizar o uso de declaração de variável dentro de um escopo de bloco determinado, com o **let**. O uso de declaração com **var** deve ser evitado, e só utilizado caso precise realmente que a variável seja alterável em um escopo geral.

1.3. Nomes

O nome de uma variável deve ser uma explicação exata sobre o valor que ela carregará, de forma que a ela se torna uma documentação de si mesma.

```
var d // tempo decorrido em dias
VS
```

```
let dtTempoDecorridoEmDias
```

1.4. Use nomes pronunciáveis

Se você não consegue pronunciar, dificilmente irá lembrar.

```
var aptVnd
VS
let bApartamentoVendido
```

1.5. Use nomes buscáveis

Se uma variável ou constante deve ser vista ou usada em múltiplas partes do código, é imperativo que ela tenha um nome facilmente buscável.

```
For (int i = 0; i < 53; i++)
VS
For (let iSemana = 0; iSemana < SEMANAS_NO_ANO; iSemana++)
```

1.6. Use uma língua consistente

Escolha uma língua e use UMA língua para dar nomes às coisas. Por exemplo, misturar Inglês e Português para dar os nomes será inconsistente e ilegível.

1.7. Não existe tamanho ideal para os nomes das Variáveis

Use nomes pequenos e longos o bastante para nomear cada coisa em cada escopo.
Apesar pequenos, nomes enigmáticos de variáveis devem ser evitados em favorecimento aos nomes longos e mais descritivos; o que não significa que devamos usar uma frase inteira. Nomes extremamente grandes são ineficientes e podem deixar o código sujo, difícil de ler/escrever.

1.8. Convenções que serão usadas.

Snakecase:
As palavras são delimitadas por um underscore: `primeiro_nome`.
Pascalcase:
As palavras são delimitadas por letas maiúsculas: `sPrimeiroNome`.
Camelcase:
As palavras são delimitadas por letas maiúsculas, a não ser a primeira: `primeiroNome`.
***Vamos dar prioridade ao Pascalcase**

1.9. Como usar no código.

Variáveis:
Serão declaradas em **Pascalcase**

```
let sPrimeiroNome = 'Agiw'
```

Contantes:
Serão declaradas em **LETRAS MAIÚSCULAS:**

```
const MINUTOS = 60
```

Booleanos:
Um Prefixo como 'is', 'are', 'has' (em português 'e', 'esta', 'tem') ajuda a distinguir um booleano de outro tipo de variável só de ver o código

```
let bEstaVisivel = true
let bTemChave = false
```

2. Funções

Os nomes das funções são escritos em **camelCase.** Sempre comece o nome da função com o verbo que defina o que está fazendo, em conjunção com o nome da **entidade** afetada pela função.

```
getOrder()
recuperaPedido()
conectaAoBanco()
```

Se a função for utilizar em outro local, **NUNCA** ter a mesma em mais de um local, e sim centralizar e exportar a mesma **GLOBALMENTE.**

3. Métodos

Assim como as funções os métodos vão ser escritos em **camelCase.** Como o nome da classe em si representa a Entidade, pós-fixar o nome da entidade no nome da função não agrega muito valor.

```
Class User {
   delete(id) {
         // Deleta usuário com id
   }
}
var user = new User();
```

4. Classes Auto Descritivas

Uma Classe deve ser declarada com **PascalCase** no seu próprio arquivo.

```
Class Usuario {
   constructor(primeiroNome, ultimoNome) {
         this.primeiroNome = primeiroNome;
         this.ultimoNome = ultimoNome
   }
}
```

5. Componentes <>

Componentes também são declaradas em **PascalCase**. Quando um componente é usado, ele é distinguido de um componente nativo em HTML, porque a primeira letra é sempre maiúscula.

```
function UserProfile(user) {
   Return (
         <div>
               <span>First Name: {user.firstName}</span>
               <span>Last Name: {user.LastName}</span>
         </div>
   )
}
<div>
<UserProfile
   User={{ firstName: 'John', lastName: 'Doe' }}
/>
</div>
```

6. Bons Argumentos

Use argumentos autoexplicativos que indiquem de forma clara qual é o propósito.

```
function calcularRestante(x, y) {}
vs
function calcularRestante(numero, divisor) {}
```

7. Propriedades Privadas

Propriedades privadas e protegidas começaram com underscore.

```
private _prisma: prisma_nest
```

8. Chaves

Será usada a convenção **K&R Style Parentheses,** isso ajudara na economia de linhas.

```
if (bTemChave) {
   // Faz alguma coisa
},
```

9. Componentes, Ui-Kit e FrameWorks

1. Componentes.
    a. Verificar qual a licença.
        i. Priorizar sempre os Gratuitos.
        ii. Se for pago passar informações para Gerência (Valor, Motivo do Uso e porque não usar uma outra).
    b. Utilizar o site do NPM como Base de Pesquisa para novos componentes. (https://www.npmjs.com/)
    c. Verificar quantidade de usuários.
    d. Se é bem recomendado pela Comunidade.
    e. Compatibilidade com a versões em uso ex: (Node, Angular, TypeScript e etc...)
2. Ui-Kit
    a. Angular-Material.
    b. Bootstrap.
    c. PrimeNg.
    d. Ionic.
3. FrameWork
    a. Back-end (NestJs e Nodejs).
    b. Front-end (Ionic e Angular).
    c. ORM (PrismaIo)

10. Novos Projetos

No commit inicial criar um arquivo README na raiz do Projeto e usar o Padrão
\\192.168.1.2\Desenvolvimento\Padrões-Manuais\README.MD

11. Ferramentas de Auxílio

Aqui serão informadas ferramentas que vão auxiliar o desenvolvimento.

1. Criação de E-mail e Interfaces em HTML.
   Bootstrapemail
   Quicktype

2. Geração de Números de Documentos (CNPJ, CPF, RG, IE e Etc...).
   4Devs
3. Criação de Componentes e estilos (CSS).
   a. Css.
      css-generator
      developer.mozilla
   b. Componentes.
      uiverse
      whirl.netlify
   c. Animações.
      animate.style
   d. Imagens.
      storyset
      freepik

12. Conventional de Commits.

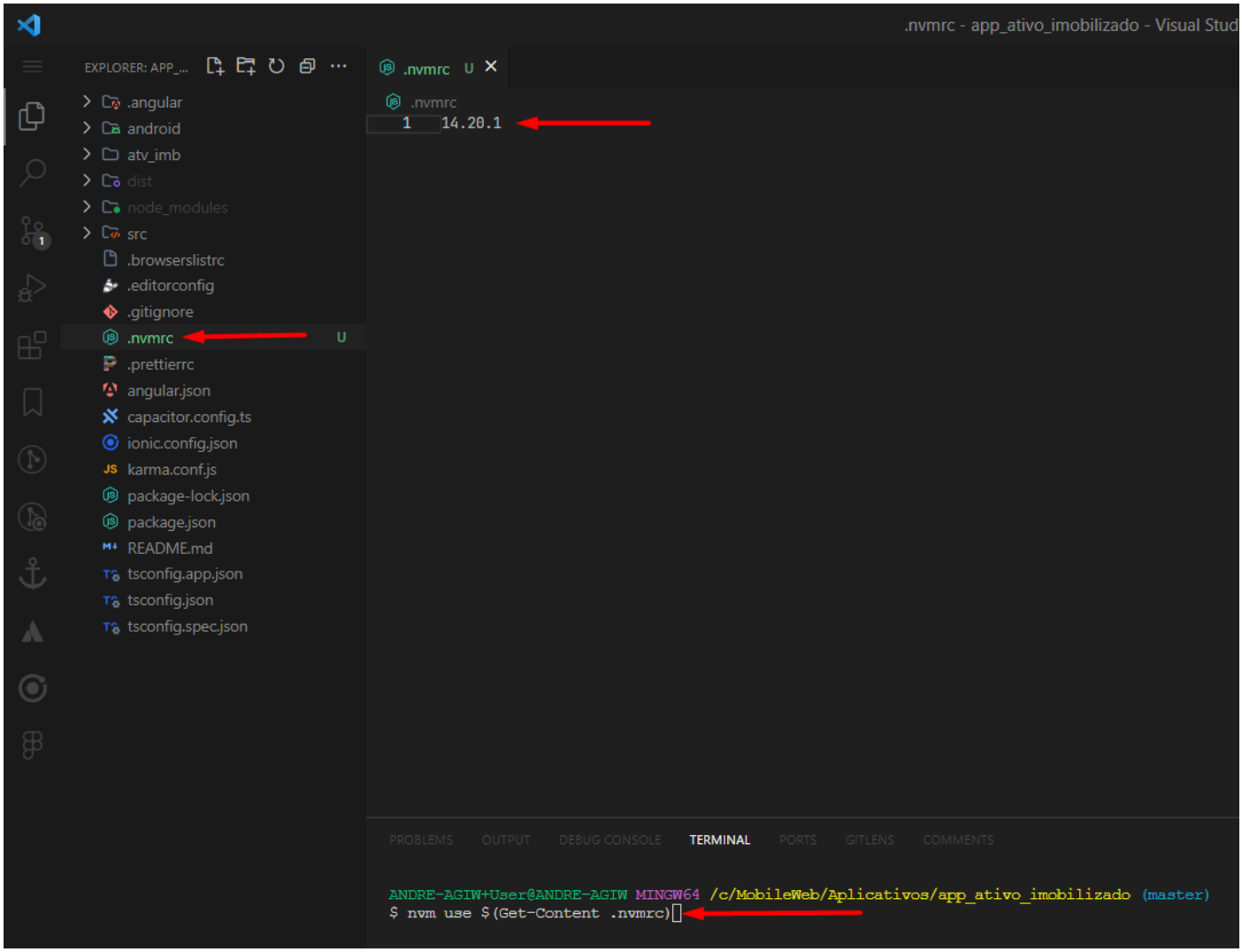
- Conventional de commits é um padrão de escrita de mensagens de commit que segue uma estrutura específica para facilitar a geração de changelogs automatizados, melhorar a comunicação entre desenvolvedores e ajudar a manter o histórico do repositório organizado.
- Os motivos do porquê são importantes usar a Convertional de Commits.
  - *Criação automatizada de CHANGELOGs*
  - *Histórico estruturado*
  - *Facilita a criação de relatórios*
  - *Facilita a entrada de novos desenvolvedores*
  - *Facilita a comunicação entre os desenvolvedores*
- **Tipos**
  - Os prefixos em "Conventional Commits" são labels utilizadas para identificar o tipo de mudança feita em um commit. O objetivo dos prefixos é fornecer uma maneira padronizada de identificar o tipo de mudança em um repositório, o que pode ser útil para equipe. Aqui você pode conferir alguns dos tipos existentes:
    - **Feat**
      Cria um novo recurso no código.
    - **Fix**
      Corrige um Bug no código.
    - **Style**
      Mudanças que não afetam o significado do código (espaço em branco, formatação falta de ponto-e-vírgula etc.).
    - **Refactor**
      Uma alteração que não corrige um bug, nem adiciona um novo recurso.
    - **Test**
      Adicionando testes ausentes ou corrigindo testes existentes.
    - **Doc**
      Apenas Documentação.
    - **Chore**
      Alterações no processo de compilação ou ferramentas e bibliotecas auxiliares, como geração de documentação.
    - **Perf**
      Mudança de Código que melhora performance.

- **Build**
  Alterações que afetam o sistema de compilação ou dependências externas (npm, npx ...).
- **Ci**
  Alterações em nossos arquivos e scripts de configuração de CI (Travis, Circle, github actions).

- **Exemplo de como vai ficar**
  - Temos que ler um commit assim "**Se eu aplicar esse commit vai**" + "Adicionar Suporte ao Inglês"
  - Ai a mensagem do Commit ficaria Feat(**Ticket**):Adicionar Suporte ao Inglês.

13. Uso do NVM por projeto.

- Ao Assumir um projeto verificar se ele possui o arquivo **.nvmrc**
  - **NÃO –** Criar o arquivo na raiz do projeto



- Após é só executar o comando `nvm use $(Get-Content .nvmrc)`

14. README.md.



README.md

15. Comentários de alteração, para rastreabilidade

Para facilitar a rastreabilidade de alterações feitas no código, bem como diferenciar alterações de correção de alterações automáticas de padronização de código do eslint, nos blocos de código alterados devemos sempre adicionar um comentário com nome do desenvolvedor, data e origem da alteração (numero da DEF ou do ticket).

```
})
morgan.token('cpuUsage', () => '')//Lucas 28/11/2023 DEF9133
app.use((req, res, next) => {
  req.id = Math.ceil(Math.random() * (9999 - 1000) + 1000)
```